

Course Project Milestone4

Automated Warehouse Scenario

Yuh-Chen Liu

Arizona State University
yliu927@asu.edu

Abstract

The Automatic Warehouse Scenario considers a perfect situation where robots deliver goods automatically based on the orders. In this paper, we would like to address our attempt to solve this issue with Answer Set Programming language, or we further shortened to ASP. Our goal is to minimize the actions of the robots for optimized solution.

Problem Statement

Automatic Warehouse Scenario describes a situation for several robots located in a limit-space rectangular grid warehouse to deliver specific products on the shelves to the picking-stations by specific orders.

The robots are designed to be flat so they can go underneath the shelves and lift the shelves and move them to the designated picking station. The robots are designed to have the following movements, and at any state of these movements, collision of two robots must be avoided:

- Move horizontally or vertically to adjacent grid cells.
- Pick up and put down shelves.
- Deliver products.
- Do nothing.

There are also the following objects with its own definition or constraints that warehouse is consist of:

- A *node* is the basic unit of the warehouse, every object that is in the warehouse is on top of a node.
- A *highway* is a unit that no shelves can be placed on.
- A *picking station* is a unit where robots deliver the products on the shelves to.
- A *shelf* is a unit where is used to stock the products waiting to be delivered.
- A *product* is located on a shelf waiting for delivery.

The goal is to generate a schedule of robot movements that minimize the time cost to complete the orders. To summarize the problem:

- Robot can only have one movement during a time unit.

- Robot needs to carry shelves to picking station following the warehouse constraints.
- Robot needs to fulfill the orders with fewest movements.

Project Background

This project statement is based on *ASP challenge 2019*, thus in this project, we selected *Clingo*, one of the ASP languages introduced in the course, as our primary programming language to solving this issue. *Clingo* is an *Answer Set Solver* developed by the *Potsdam University* and has been widely practiced during this course.

During the course project milestones, we have successfully setup *Clingo* in our local machines and was able to solve simple answer set problems with *Clingo* as an answer set solver. We further had our first glance at the automatic warehouse problem in the previous project milestone, where we dissected our tasks into smaller working items base on the project requirements and listed out the issues we were facing and the approaches we planned on to tackle those issues. From our last results, we managed to complete a simple transition system, which is functionated to move robots freely under warehouse constraints and we were working on the mechanism of having robots moving the shelves to designated positions.

We also revisited Dr.Lee's lecture slides regarding the complex transition systems. In the lecture, Dr.Lee introduced several important concepts when designing a complex transition system. Here we would like to walk through these important concepts which we later referred frequently in the project. According to Dr.Lee:

A *transition system* is a direct graph whose vertices correspond to the state of the world. Its edges represent execution of actions. (Lee 2016)

Within the lecture, we have acquired the knowledge that a *Fluent* represents anything that depends on the state of the world, and *the commonsense of inertia* describes that by

default, the state of a *fluent* remains unchanged after executing actions. These are concepts that can be transformed into *Clingo* program when implementing a transition system. We then derived the knowledge from the lecture to know that a transition system is composed by *fluent*, the *commonsense of inertia*, *actions*, and *environmental constraints*, which are based on the given circumstance and the use cases by the project.

Given the experience from our previous milestones and attempts, we were able to setup the *fluent* of robots, the *commonsense of inertia* of robot position and the *direct effect* of robot movements. We have also concluded that an *action* is composed by the rules of *action being exogenous*, the *direct effect of actions*, and *constraints of the actions*. Later in this project, we will be implementing more *fluent* like product status, shelf location and orders status. More *robot actions* like pickup, put down, and deliver, and the *commonsense of inertia* for all the *fluent* we mentioned above. Given the use cases we also have *constraints* like robot cannot run into each other, robot cannot go under shelves if it's already carrying one, etc.

Approach

ASP language is not easy to debug, for most of the time the issue of not finding a stable model or having a stable model with undesired results is not because there is a syntax error that disrupts the code, but because there is a logic error in the rules we add. Given from the project background we understand that there are certain formats of these rules that we can make use of. During the project development, we generated the *fluent*, the *commonsense of inertia* and the *uniqueness and existence of value constraints* for all the *fluent* based on the same approach in Dr.Lee's lecture slides. We then narrow our working items into the looping process of:

- **Introduce a new action.**
- **Test the direct effect of the action.**
- **Review and debug rules of the action if it is not working as expected**
- **Add constraints based on use cases of the actions.**
- **Test expected action movements based on the optimal plan provided in the project description.**
- **Review and debug rules of the action if it is not working as expected**

We strictly followed the optimized plan in project description by stating them as testing scripts. During our previous attempt when conducting robot movements, we started our testing plan with appointing the robot to be in certain location. The script then followed by the optimized plan to ask the robot to do the same movement as the plan did. This approach has been replicated several times when dealing with shelves, products and orders, and this approach has

served us well as we managed to generate the same optimized plan in the project description.

Main Results and Analysis

Our implementation managed to achieve the same optimized plan as the one in the description plan in around one minute on our local machine with the minimal time unit 13. We match the movements from the project description with our execution results and found them identical, which gave us confidence to say our implementation has fulfilled most of the project requirements. We further ran through all the example using one thread with *Clingo* and reach the following results:

- Instance1, CPU time: 71, Optimization: 13
- Instance2, CPU time: 34, Optimization: 11
- Instance3, CPU time: 0.1, Optimization: 7
- Instance4, CPU time: 1.15, Optimization: 10
- Instance5, CPU time: 0.06, Optimization: 6

We further attempt to analyze if our optimized plan for instance5 is correct, as it has fewer steps, thus easier for validation. In Instance5, we have only one order, which includes delivering 1 Product1 and 4 Product3 to the only picking station, where Product1 is located on Shelf3 with 1 count and 4 Product3 on Shelf6. In this scenario, because Shelf3 locates at (2, 3) and Shelf6 locates at (1, 2) with picking station locates at (1, 3), which means both of our object shelves are at the adjacent cells of the picking station. We then find our robots at coordination (4, 3) and (2, 2), which means Robot1 will move to its left twice, spending two time-units until it reaches Shelf3 (2, 3) and Robot2 will move to its left once to Shelf6 (1, 2) at time 1. We then assumed the Robot1 will pick up Shelf3 and deliver it to the picking station and Robot2 lifting Shelf6 to the picking station. Robot2 will pick up Shelf6 at time 2, move to the picking station at time 3, deliver at time 4. Robot1 will be picking up Shelf3 at time 3, but Robot1 cannot move to the picking station at time 4, otherwise it would collide with Robot2 in delivery, so Robot1 can only move to picking station at time 5, deliver at time 6. We compared this presumed result with our results running by *Clingo*, here we list out the movements to prove it has followed our presumed optimized results:

- occurs(object(robot,1),move(-1,0),1)
- occurs(object(robot,2),move(-1,0),1)
- occurs(object(robot,1),move(-1,0),2)
- occurs(object(robot,2),pickup,2)
- occurs(object(robot,1),pickup,3)
- occurs(object(robot,2),move(0,1),3)
- occurs(object(robot,2),deliver(1,3,4),4)
- occurs(object(robot,1),move(-1,0),5)
- occurs(object(robot,2),move(0,-1),5)

- occurs(object(robot,1),deliver(1,1,1),6)
- occurs(object(robot,2),putdown,6)

Given the results in Instance1 and Instance6, we have confidence to say that our implementation fulfills the project requirements, and the implementation result is a success.

Conclusion

In this project, we have successfully implemented a simulated scenario of an automatic warehouse under an ideal condition. During the development of this project, we have made a nice approach to divide the requirements into small working items and conquer each segment with the functions tested by each phase. We have budgeted our time well with the total development time by estimate, 48 hours from the start of development of this project to the phase we have decided we have fulfilled the project requirement. However, we also spent one third of the development time on the same issue of timestamp with actions because we did not analyze and identify the fluent and the rules suitable for them well enough in our previous attempt. We had to adjust our approach from the previous milestone and reinitiate our attempt to solve the issue, because we found that trying to fix a set of poor written rules is even harder than starting from scratch, thus conclude the biggest challenge we faced during the project development, the semantic of ASP is not easy to interpret and the logic errors are harder to detect based on our current proficiency in the language.

Knowledge Representation and Reasoning is a wide research field in Artificial Intelligence that scientist has been working on for decades. From this project, we have experienced one of the powerful products from their research known as ASP and have utilize one of the ASP solvers known as *Clingo* to solve a simulated real-life problem. This project is a simplified practice of the actual problem we would like to solve or contribute to solving in the future, in real-life, there might be more issues and constraints we would like to reason with. Nevertheless, this project provides a primarily approach into solving the actual automatic warehouse issue.

Opportunity for Future Work

We further investigate the project and think about how we can improve our program to be more applicable to be utilize in an actual automatic warehouse. Currently we have assumed that the robots have infinite fuels or power, so we do not have to worry about battery level or navigate to the recharge spot. In real-life, the orders come in real-time, while currently we just assumed everything is predefined. If we would like to adjust the instance based on the consistently changing real-life circumstance, it is without

doubt that the instance file needs to be generated by traditional programming language and call upon the solver to look for answer. For a planning server in the warehouse sending commands to the robots, the possibility of integrating with traditional programming language is a must. Given the discussion that the solve might be the server to send out the signal, the performance of our current rules must be improved. By ruling out some edge cases and put them in the constraints, the likelihood of performance boost is also promising.

References

- Technische Universität Wien (Austria) and the University of Genoa (Italy), *Answer Set Programming Challenge (2019)*, <https://sites.google.com/view/aspcomp2019>
- Lee, J and Bartholomew, M. 2016. AAAI 2016 Tutorial *Answer Set Programming Modulo Theories*, Arizona State University.

Appendix

```
% init objects
node(N,X,Y) :- init(object(node,N), value(at,pair(X,Y))).
highway(N,X,Y) :- init(object(highway,N), value(at,pair(X,Y))).
pickingStation(P,X,Y) :- init(object(pickingStation,P), value(at,pair(X,Y))).
object(robot,R) :- init(object(robot,R), value(at,pair(X,Y))).
object(shelf,S) :- init(object(shelf,S), value(at,pair(X,Y))).
orderTo(O,P) :- init(object(order,O),value(pickingStation,P)).
orderContent(O,I,C) :- init(object(order,O),value(line,pair(I,C))).

%fluent
loc(robot,R,X,Y,0) :- init(object(robot,R), value(at,pair(X,Y))).
loc(shelf,S,X,Y,0) :- init(object(shelf,S), value(at,pair(X,Y))).
productLoc(I,S,C,0) :- init(object(product,I), value(on,pair(S,C))).
orderComplete(O,I,C,0,0) :- init(object(order,O),value(line,pair(I,C))).
on(0,S,0) :- init(object(shelf,S), value(at,pair(X,Y))).

% uniqueness and existence of value constraints.
:- not {loc(robot,R,X,Y,T):node(N,X,Y)}=1, object(robot,R), T=1..m.
:- not {loc(shelf,S,X,Y,T):node(N,X,Y)}=1, object(shelf,S), T=1..m.
:- not {productLoc(I,S,C,T)}=1, productLoc(I,S,C,T), T=1..m.
:- not {orderComplete(O,I,C,U,T)}=1, orderContent(O,I,C), T = 1..m.
:- not {on(_S,T)}=1,object(shelf,S), T = 1..m.

% commonsense law of inertia
{loc(robot,R,X,Y,T+1)} :- loc(robot,R,X,Y,T), T=0..m-1.
{loc(shelf,S,X,Y,T+1)} :- loc(shelf,S,X,Y,T), T=0..m-1.
{productLoc(I,S,C,T+1)} :- productLoc(I,S,C,T), T=0..m-1.
{orderComplete(O,I,C,U,T+1)} :- orderComplete(O,I,C,U,T), T=0..m-1.
{on(R,S,T+1)} :- on(R,S,T),T=0..m-1.

% constraints
% 1 action at a time
:- 2 {occurs(object(robot,R),X,T)}, object(robot,R), T = 1..m.
% two shelves cannot collide
:- loc(shelf,S,X,Y,T), loc(shelf,S1,X,Y,T), S!=S1.
% avoid collision between robots
:- loc(robot,R,X,Y,T), loc(robot,R1,X,Y,T), R!=R1.
% force product loc exist
:- not {productLoc(I,S,_T)}=1, productLoc(I,S,_T-1), T=1..m.

%move
```

```
move(1,0; -1,0; 0,1; 0,-1).
% robot movement is exogenous
{occurs(object(robot,R), move(X1,Y1),T)} :- object(robot,R), move(X1,Y1), T=1..m.
% direct effect.
loc(robot,R,X+X1,Y+Y1,T) :- occurs(object(robot,R), move(X1,Y1), T), loc(robot,R,X,Y,T-1), T=1..m.
%bots are not allowed to switch places.
:- loc(robot,R,X1,Y1,T), loc(robot,R1,X2,Y2,T), loc(robot,R,X2,Y2,T+1), loc(robot,R1,X1,Y1,T+1), R!=R1.
%robot cannot step outside of warehouse.
:- not node(_X,Y), loc(robot,R,X,Y,T), T=1..m.
% pickup is exogenous
{pickup(R,S,T): object(shelf,S)} 1 :- object(robot,R), T=1..m.
occurs(object(robot,R), pickup,T) :- pickup(R,S,T).
% concept of on
on(R,S,T) :- pickup(R,S,T).
% direct effect
loc(shelf,S,X,Y,T) :- on(R,S,T), loc(robot,R,X,Y,T).
% can only pickup at same location
:- pickup(R,S,T), loc(robot,R,X,Y,T-1), not loc(shelf,S,X,Y,T-1).

% cannot pickup two shelves
:- pickup(R,S,T), on(R,S1,T-1), T = 0..m.
% putdown is exogenous
{occurs(object(robot,R), putdown,T)} 1 :- object(robot,R), T=1..m.
%direct effect
on(0,S,T) :- occurs(object(robot,R), putdown,T), on(R,S,T-1), T=1..m.
% cannot do putdown if is not lift.
:- not on(R,_T-1), occurs(object(robot,R), putdown,T), T=1..m.
:- highway(N,X,Y), loc(shelf,S,X,Y,T), loc(robot,R,X,Y,T), not on(R,S,T).
% product can't < 0
:- productLoc(P,S,C,T), C < 0, T = 1..m.
% only deliver exact order count
:- orderComplete(O,I,C,U,T), C < U, T = 1..m.

%deliver is exogenous
%{occurs(object(robot,R),deliver(O,I,U),T)} 1 :- object(robot,R), on(R,S,T-1), productLoc(I,S,U,T-1), orderComplete(O,I,C,_T), T=1..m.
{deliver(R,O,I,U,T)} 1 :- object(robot,R), on(R,S,T-1), productLoc(I,S,_T-1), orderComplete(O,I,C,U1,T-1), U=C-U1, T=1..m.
occurs(object(robot,R),deliver(O,I,U),T) :- deliver(R,O,I,U,T).
% picking station
:- deliver(R,O,I,U,T), loc(robot,R,X,Y,T-1), orderTo(O,P), not pickingStation(P,X,Y).
% shelf needs to be on
:- deliver(R,O,_,_T), not on(R,_T-1).
% need to carry valid product
```

```
:- deliver(R,O,I,_,T), object(robot,R), on(R,S,T-1), not product-  
Loc(I,S,_,T-1).
```

```
%direct effect
```

```
orderComplete(O,I,C,U+U1,T) :- deliver(R,O,I,U,T), orderCom-  
plete(O,I,C,U1,T-1).
```

```
productLoc(I,S,C-U,T) :- deliver(R,O,I,U,T), on(R,S,T-1),  
productLoc(I,S,C,T-1).
```

```
%goal
```

```
:- orderComplete(O,I,C,U,m), C!=U.
```

```
#minimize{1,T: occurs(object(robot,_,_),T)}.
```

```
#show occurs/3.
```